

How the Pipeline Works

Interview → Route → Fetch / Build / Adapt → Validate

Talk to Galaxy · `dejian_workflow/` · NIAID-BRCs AI Codeathon 2.0

The one-line version

A free-text research description goes in. A real, mechanically-validated Galaxy workflow file comes out — reused, adapted, or built from scratch, never just a plausible-looking guess.



Driven end-to-end by `scripts/run_pipeline.py`.

Stage 1 — Interview

A plain text file, in the researcher's own words:

"I have Illumina paired-end amplicon sequencing of SARS-CoV-2 samples and want variant calls and lineage assignment with pangolin."

No structured form, no dropdown of organisms — this is deliberately the messiest input in the pipeline, because that's what a real interview produces. Three examples ship in `interviews/raw/` to test with immediately.

Stage 2 — Route (`route.py`)

```
route.py
└─ search_pipeline_catalog.py
    └─ knowledge_base/pathogen_genomics/galaxy_pipeline_catalog.yaml
        (20 curated bacteria/virus IWC workflows)
```

1. Tokenize the description
2. Score every catalog entry by **IDF-weighted keyword overlap** — a rare, specific term ("cgmlst," "pangolin") counts far more than a common one ("illumina," "paired")
3. Take the top match's **confidence** (0–1) and threshold it:

Confidence	Decision
≥ 0.35	use — strong, specific match
0.15 – 0.35	adapt — plausible but partial match
< 0.15 / no match	build — nothing close enough in the catalog

Why IDF weighting, not raw keyword counts?

Raw overlap counts let generic terms dominate. A vague request sharing "illumina," "paired," and "end" with ten different workflows would outscore a precise one.

```
"...SARS-CoV-2 samples... lineage assignment with pangolin"  
→ matched: amplicon, assignment, cov, end, illumina,  
            lineage, paired, pangolin, sars, variant  
→ confidence 0.50    (dominated by rare terms: pangolin, lineage, sars)
```

The catalog entry's own confidence is *matched-weight ÷ that entry's total keyword weight* — comparable across entries with different-length keyword lists, not just a raw count.

Stage 2b — Follow-up questions (Goal 4)

```
route.py → generate_followup_questions.py
```

For each routed candidate: did the description actually confirm organism / data_type / analysis_type, or is it a silent assumption?

Anything with **zero matched keywords** becomes a question, not a guess.

Real example (E. coli cgMLST request, routed to "adapt", confidence 0.21):

1. assumes **bacteria** — description didn't confirm
2. assumes **assembly/FASTA** — description didn't confirm
3. assumes **strain typing** — description didn't confirm
4. only a partial match — what's actually different?

Deterministic, not LLM-based — needs no credentials, only surfaces gaps that provably exist in the matching data, never fabricated concerns.

Known limit: surfaces questions, doesn't collect answers or re-route on them — the interview loop that closes this isn't built yet.

Stage 3a — Use path

```
fetch_iwc_workflow.py → validate_workflow.sh
```

- Lists the workflow's real directory on GitHub via the API (IWC has **no consistent filename convention** — guessing from the directory name silently fails on ~1/3 of workflows)
- Downloads the actual `.ga` + Planemo test file
- Runs `planemo workflow_lint` — checks tool ids, version/changeset installability, and step connections against the **live Galaxy Tool Shed**, no Galaxy server required

This already caught a real bug: a published SARS-CoV-2 workflow in IWC currently references an uninstallable toolshed changeset revision.

Stage 3b — Adapt path

Fetches the same real base workflow, then stops — deciding *what* to change from free text needs a human or an LLM, and this script doesn't fabricate that decision. Instead it takes a **change spec**:

```
[{"step_id": "2",  
  "tool_state_path": ["scannew_section", "min_id_new_allele"],  
  "new_value": "95",  
  "note": "researcher wants a stricter identity threshold"}]
```

```
adapt_workflow.py propose workflow.ga change_spec.json # review the diff  
adapt_workflow.py apply workflow.ga change_spec.json out.ga
```

`apply` deep-copies the workflow, edits only the named steps, then **byte-compares every other step against the original** — refuses to write output if anything outside the change spec differs.

Stage 3c — Build path

No workflow in the catalog is close enough → generate one.

```
build_workflow.py
  ↳ anthropic SDK (claude-opus-5)
    ↳ validate_workflow.sh (same real check as the use path)
```

- The LLM is told to only use tool ids it's genuinely confident exist
- Generation is **not trusted on its own** — the output goes through the exact same `planemo workflow_lint` check as a fetched workflow
- A generated workflow that fails validation is reported as a **failure**, not silently written as "done"

This is the one path most exposed to confident fabrication — which is exactly why it's graded by the same program as everything else, not exempt from it.

Stage 4 — Test execution (separate step)

```
run_workflow_tests.sh workflow.ga
```

Runs `planemo test` — executes the workflow against real data, not just structural checks.

Mode	Where it actually executes
<code>GALAXY_URL</code> + <code>GALAXY_USER_KEY</code> (preferred)	Remote — e.g. usegalaxy.org's own compute
<code>GALAXY_ROOT</code>	Local — a Galaxy checkout on your machine
neither set → <code>--install_galaxy</code>	Local — a disposable Galaxy Planemo downloads

Static validation (stage 3) only checks the workflow is *well-formed*. This is the step that checks it actually *does the right analysis*.

Where does the data actually go?

This is not just a speed tradeoff — it's a data-location decision.

- **Online (`GALAXY_URL` / `GALAXY_USER_KEY`)**: your machine only orchestrates. The workflow and test data are **uploaded to that Galaxy server**, tools run on **its compute**, and results are downloaded back. Fast, zero local setup — but your data leaves your machine and is processed by a third-party public service.
- **Local (`GALAXY_ROOT` or `--install_galaxy`)**: Galaxy and every tool run **entirely on your machine**. Nothing is uploaded anywhere. Slower to set up, but your data never leaves.

Rule of thumb: fine for the example/test data this pipeline ships with.

For real, sensitive, or restricted research data, use the local mode.

What's real vs. what's a stand-in

Piece	Status
Catalog, routing, fetch, static validation, byte-stable adapt	Real , tested against live GitHub + Galaxy Tool Shed
Follow-up questions	Real, deterministic — surfaces gaps, but no answer-collection loop yet
Build path	Real LLM call + real validation, but no live Galaxy Workflow Foundry integration
Adapt path's "what to change"	Needs a human/LLM-authored change spec — no NL interpretation yet
Catalog coverage	~20 curated workflows, not the full IWC registry

None of this is faked to look more finished than it is — see each script's own docstring and `SETUP.md` for the honest gaps.

Try it

```
cd dejian_workflow  
pip install -r requirements.txt  
python3 scripts/run_pipeline.py interviews/raw/example-sarscov2-amplicon.txt  
python3 scripts/run_pipeline.py interviews/raw/example-ecoli-outbreak-cgmlst.txt
```

Full install/credentials/cleanup instructions: `SETUP.md`

Script-by-script reference: `scripts/README.md`

Full project proposal: `PROPOSAL.md`